Contours : Getting Started

Next Tutorial: Contour Features

Goal

- Understand what contours are.
- Learn to find contours, draw contours etc
- You will see these functions : **cv.findContours()**, **cv.drawContours()**

What are contours?

Contours can be explained simply as a curve joining all the continuous points (along the boundary), having same color or intensity. The contours are a useful tool for shape analysis and object detection and recognition.

- For better accuracy, use binary images. So before finding contours, apply threshold or canny edge detection.
- Since OpenCV 3.2, findContours() no longer modifies the source image.
- In OpenCV, finding contours is like finding white object from black background. So remember, object to be found should be white and background should be black.

Let's see how to find contours of a binary image:

```
import numpy as np
import cv2 as cv

im = cv.imread('test.jpg')
```

```
assert im is not None, "file could not be read, check with os.path.exists()"
imgray = cv.cvtColor(im, cv.COLOR_BGR2GRAY)
ret, thresh = cv.threshold(imgray, 127, 255, 0)
contours, hierarchy = cv.findContours(thresh, cv.RETR_TREE, cv.CHAIN_APPROX_SIMPLE)
```

See, there are three arguments in **cv.findContours()** function, first one is source image, second is contour retrieval mode, third is contour approximation method. And it outputs the contours and hierarchy. Contours is a Python list of all the contours in the image. Each individual contour is a Numpy array of (x,y) coordinates of boundary points of the object.

Note

We will discuss second and third arguments and about hierarchy in details later. Until then, the values given to them in code sample will work fine for all images.

How to draw the contours?

To draw the contours, **cv.drawContours** function is used. It can also be used to draw any shape provided you have its boundary points. Its first argument is source image, second argument is the contours which should be passed as a Python list, third argument is index of contours (useful when drawing individual contour. To draw all contours, pass -1) and remaining arguments are color, thickness etc.

- To draw all the contours in an image:

  ```
  cv.drawContours(img, contours, -1, (0,255,0), 3)
  ```

- To draw an individual contour, say 4th contour:

  ```
  cv.drawContours(img, contours, 3, (0,255,0), 3)
  ```

- But most of the time, below method will be useful:

  ```
  cnt = contours[4]
  cv.drawContours(img, [cnt], 0, (0,255,0), 3)
  ```

Note

Last two methods are same, but when you go forward, you will see last one is more useful.

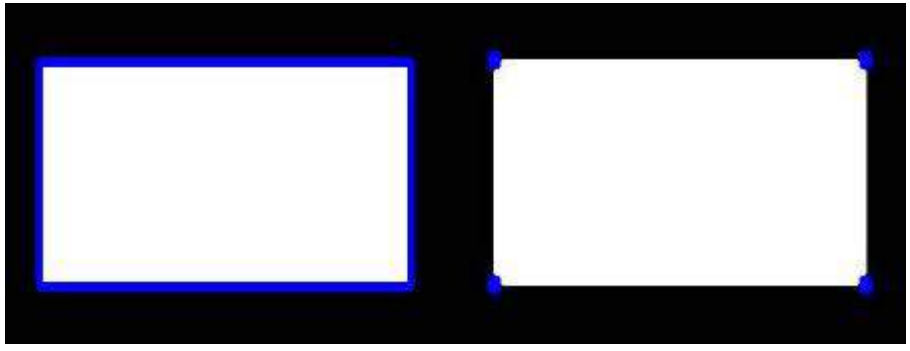
Contour Approximation Method

This is the third argument in **cv.findContours** function. What does it denote actually?

Above, we told that contours are the boundaries of a shape with same intensity. It stores the (x,y) coordinates of the boundary of a shape. But does it store all the coordinates ? That is specified by this contour approximation method.

If you pass **cv.CHAIN_APPROX_NONE**, all the boundary points are stored. But actually do we need all the points? For eg, you found the contour of a straight line. Do you need all the points on the line to represent that line? No, we need just two end points of that line. This is what **cv.CHAIN_APPROX_SIMPLE** does. It removes all redundant points and compresses the contour, thereby saving memory.

Below image of a rectangle demonstrate this technique. Just draw a circle on all the coordinates in the contour array (drawn in blue color). First image shows points I got with **cv.CHAIN_APPROX_NONE** (734 points) and second image shows the one with **cv.CHAIN_APPROX_SIMPLE** (only 4 points). See, how much memory it saves!!!



image

Additional Resources

Exercises

Contour Features

Prev Tutorial: Contours : Getting Started
Next Tutorial: Contour Properties

Goal

In this article, we will learn

- To find the different features of contours, like area, perimeter, centroid, bounding box etc
- You will see plenty of functions related to contours.

1. Moments

Image moments help you to calculate some features like center of mass of the object, area of the object etc. Check out the wikipedia page on Image Moments

The function **cv.moments()** gives a dictionary of all moment values calculated. See below:

```
import numpy as np
import cv2 as cv

img = cv.imread('star.jpg', cv.IMREAD_GRAYSCALE)
assert img is not None, "file could not be read, check with os.path.exists()"
ret,thresh = cv.threshold(img,127,255,0)
contours,hierarchy = cv.findContours(thresh, 1, 2)

cnt = contours[0]
```

```
M = cv.moments(cnt)
print( M )
```

From this moments, you can extract useful data like area, centroid etc. Centroid is given by the relations, $C_x = \frac{M_{10}}{M_{00}}$ and $C_y = \frac{M_{01}}{M_{00}}$. This can be done as follows:

```
cx = int(M['m10']/M['m00'])
cy = int(M['m01']/M['m00'])
```

2. Contour Area

Contour area is given by the function **cv.contourArea()** or from moments, **M['m00']**.

```
area = cv.contourArea(cnt)
```

3. Contour Perimeter

It is also called arc length. It can be found out using **cv.arcLength()** function. Second argument specify whether shape is a closed contour (if passed True), or just a curve.

```
perimeter = cv.arcLength(cnt,True)
```

4. Contour Approximation

It approximates a contour shape to another shape with less number of vertices depending upon the precision we specify. It is an implementation of Douglas-Peucker algorithm. Check the wikipedia page for algorithm and demonstration.

To understand this, suppose you are trying to find a square in an image, but due to some problems in the image, you didn't get a perfect square, but a "bad shape" (As shown in first image below). Now you can use this function to approximate the shape. In this, second argument is called epsilon, which is maximum distance from contour to approximated contour. It is an accuracy parameter. A wise selection of epsilon is needed to get the correct output.

```
epsilon = 0.1*cv.arcLength(cnt,True)
approx = cv.approxPolyDP(cnt,epsilon,True)
```

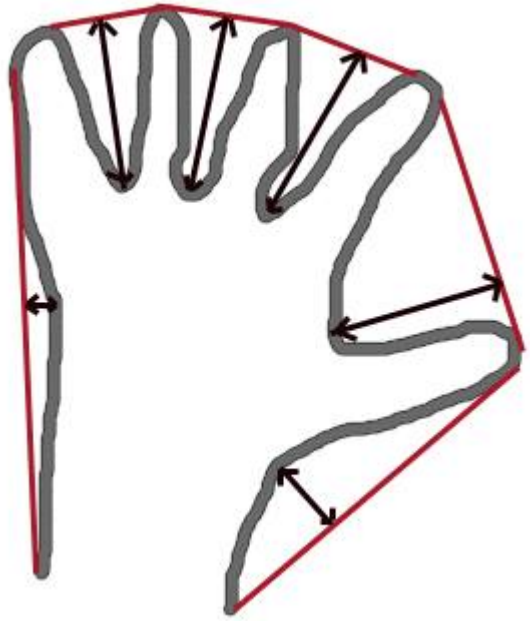
Below, in second image, green line shows the approximated curve for epsilon = 10% of arc length. Third image shows the same for epsilon = 1% of the arc length. Third argument specifies whether curve is closed or not.



image

5. Convex Hull

Convex Hull will look similar to contour approximation, but it is not (Both may provide same results in some cases). Here, **cv.convexHull()** function checks a curve for convexity defects and corrects it. Generally speaking, convex curves are the curves which are always bulged out, or at-least flat. And if it is bulged inside, it is called convexity defects. For example, check the below image of hand. Red line shows the convex hull of hand. The double-sided arrow marks shows the convexity defects, which are the local maximum deviations of hull from contours.



image

There is a little bit things to discuss about it its syntax:

```
hull = cv.convexHull(points[, hull[, clockwise[, returnPoints]]])
```

Arguments details:

- **points** are the contours we pass into.
- **hull** is the output, normally we avoid it.
- **clockwise** : Orientation flag. If it is True, the output convex hull is oriented clockwise. Otherwise, it is oriented counter-clockwise.
- **returnPoints** : By default, True. Then it returns the coordinates of the hull points. If False, it returns the indices of contour points corresponding to the hull points.

So to get a convex hull as in above image, following is sufficient:

```
hull = cv.convexHull(cnt)
```

But if you want to find convexity defects, you need to pass returnPoints = False. To understand it, we will take the rectangle image above. First I found its contour as cnt. Now I found its convex hull with returnPoints = True, I got following values: [[[234 202]], [[51 202]], [[51 79]], [[234 79]]] which are the four corner points of rectangle. Now if do the same with returnPoints = False, I get following result: [[129],[67],[0],[142]]. These are the indices of corresponding points in contours. For eg, check the first value: cnt[129] = [[234, 202]] which is same as first result (and so on for others).

You will see it again when we discuss about convexity defects.

6. Checking Convexity

There is a function to check if a curve is convex or not, **cv.isContourConvex()**. It just return whether True or False. Not a big deal.

```
k = cv.isContourConvex(cnt)
```

7. Bounding Rectangle

There are two types of bounding rectangles.

7.a. Straight Bounding Rectangle

It is a straight rectangle, it doesn't consider the rotation of the object. So area of the bounding rectangle won't be minimum. It is found by the function **cv.boundingRect()**.

Let (x,y) be the top-left coordinate of the rectangle and (w,h) be its width and height.

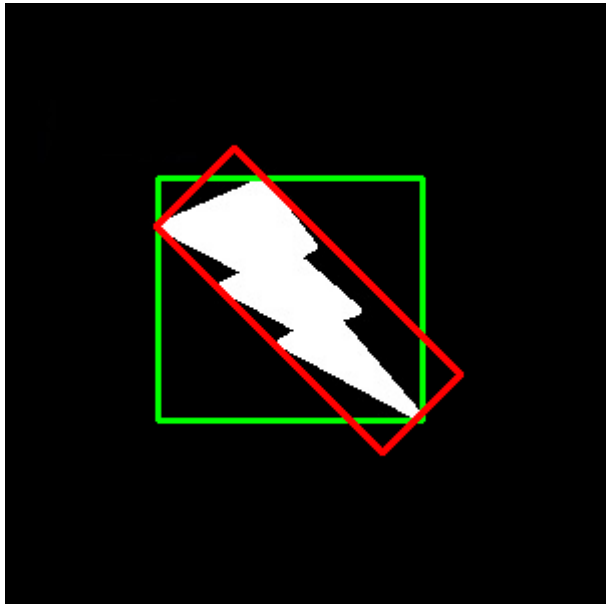
```
x,y,w,h = cv.boundingRect(cnt)
cv.rectangle(img,(x,y),(x+w,y+h),(0,255,0),2)
```

7.b. Rotated Rectangle

Here, bounding rectangle is drawn with minimum area, so it considers the rotation also. The function used is **cv.minAreaRect()**. It returns a Box2D structure which contains following details - (center (x,y), (width, height), angle of rotation). But to draw this rectangle, we need 4 corners of the rectangle. It is obtained by the function **cv.boxPoints()**

```
rect = cv.minAreaRect(cnt)
box = cv.boxPoints(rect)
box = np.int0(box)
cv.drawContours(img,[box],0,(0,0,255),2)
```

Both the rectangles are shown in a single image. Green rectangle shows the normal bounding rect. Red rectangle is the rotated rect.

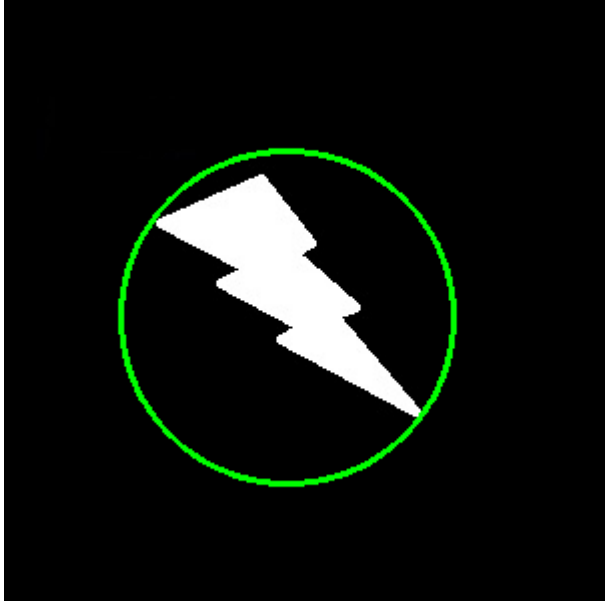


image

8. Minimum Enclosing Circle

Next we find the circumcircle of an object using the function **cv.minEnclosingCircle()**. It is a circle which completely covers the object with minimum area.

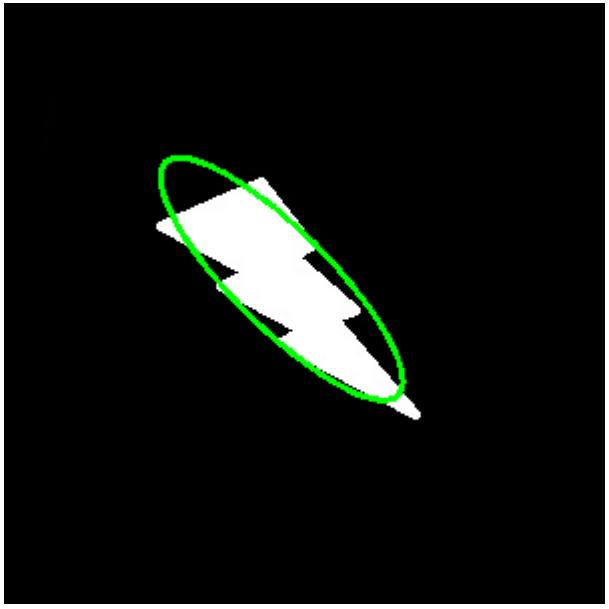
```
(x,y),radius = cv.minEnclosingCircle(cnt)
center = (int(x),int(y))
radius = int(radius)
cv.circle(img,center,radius,(0,255,0),2)
```



9. Fitting an Ellipse

Next one is to fit an ellipse to an object. It returns the rotated rectangle in which the ellipse is inscribed.

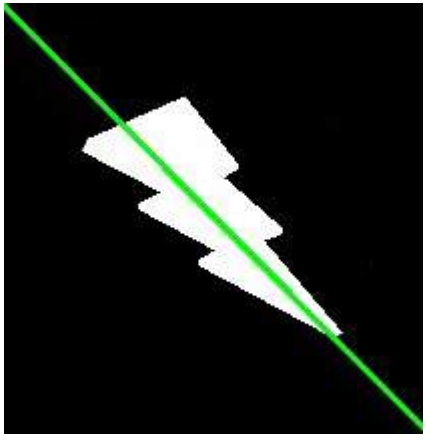
```
ellipse = cv.fitEllipse(cnt)
cv.ellipse(img,ellipse,(0,255,0),2)
```



10. Fitting a Line

Similarly we can fit a line to a set of points. Below image contains a set of white points. We can approximate a straight line to it.

```
rows,cols = img.shape[:2]
[vx,vy,x,y] = cv.fitLine(cnt, cv.DIST_L2,0,0.01,0.01)
lefty = int((-x*vy/vx) + y)
righty = int(((cols-x)*vy/vx)+y)
cv.line(img,(cols-1,righty),(0,lefty),(0,255,0),2)
```



Additional Resources

Exercises

ENHANCED BY Google

Contour Properties

Prev Tutorial: Contour Features
Next Tutorial: Contours : More Functions

Here we will learn to extract some frequently used properties of objects like Solidity, Equivalent Diameter, Mask image, Mean Intensity etc. More features can be found at Matlab regionprops documentation.

(NB : Centroid, Area, Perimeter etc also belong to this category, but we have seen it in last chapter)

1. Aspect Ratio

It is the ratio of width to height of bounding rect of the object.

$$Aspect\ Ratio = \frac{Width}{Height}$$

```
x,y,w,h = cv.boundingRect(cnt)
aspect_ratio = float(w)/h
```

2. Extent

Extent is the ratio of contour area to bounding rectangle area.

$$Extent = \frac{Object\ Area}{Bounding\ Rectangle\ Area}$$

```
area = cv.contourArea(cnt)
```

```
x,y,w,h = cv.boundingRect(cnt)
rect_area = w*h
extent = float(area)/rect_area
```

3. Solidity

Solidity is the ratio of contour area to its convex hull area.

$$Solidity = \frac{Contour\ Area}{Convex\ Hull\ Area}$$

```
area = cv.contourArea(cnt)
hull = cv.convexHull(cnt)
hull_area = cv.contourArea(hull)
solidity = float(area)/hull_area
```

4. Equivalent Diameter

Equivalent Diameter is the diameter of the circle whose area is same as the contour area.

$$Equivalent\ Diameter = \sqrt{\frac{4 \times Contour\ Area}{\pi}}$$

```
area = cv.contourArea(cnt)
equi_diameter = np.sqrt(4*area/np.pi)
```

5. Orientation

Orientation is the angle at which object is directed. Following method also gives the Major Axis and Minor Axis lengths.

```
(x,y),(MA,ma),angle = cv.fitEllipse(cnt)
```

6. Mask and Pixel Points

In some cases, we may need all the points which comprises that object. It can be done as follows:

```
mask = np.zeros(imgray.shape,np.uint8)
cv.drawContours(mask,[cnt],0,255,-1)
pixelpoints = np.transpose(np.nonzero(mask))
#pixelpoints = cv.findNonZero(mask)
```

Here, two methods, one using Numpy functions, next one using OpenCV function (last commented line) are given to do the same. Results are also same, but with a slight difference. Numpy gives coordinates in **(row, column)** format, while OpenCV gives coordinates in **(x,y)** format. So basically the answers will be interchanged. Note that, **row = y** and **column = x**.

7. Maximum Value, Minimum Value and their locations

We can find these parameters using a mask image.

```
min_val, max_val, min_loc, max_loc = cv.minMaxLoc(imgray,mask = mask)
```

8. Mean Color or Mean Intensity

Here, we can find the average color of an object. Or it can be average intensity of the object in grayscale mode. We again use the same mask to do it.

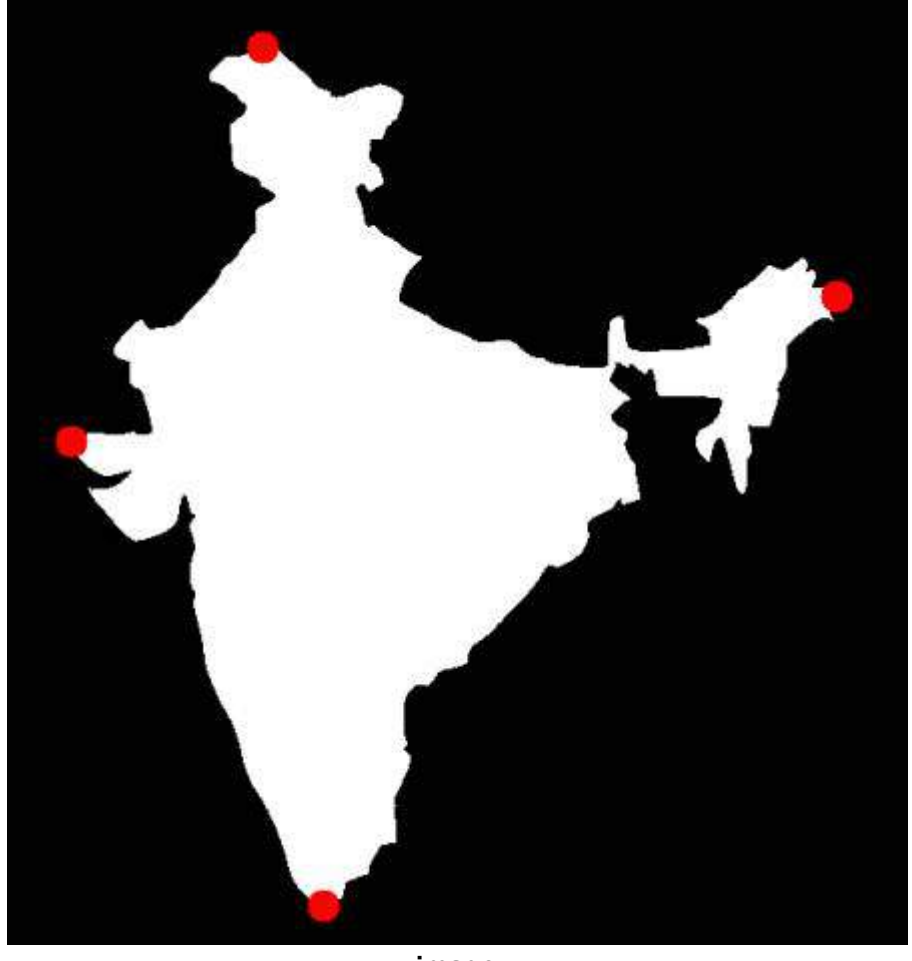
```
mean_val = cv.mean(im,mask = mask)
```

9. Extreme Points

Extreme Points means topmost, bottommost, rightmost and leftmost points of the object.

```
leftmost = tuple(cnt[cnt[:,:,0].argmin()][0])
rightmost = tuple(cnt[cnt[:,:,0].argmax()][0])
topmost = tuple(cnt[cnt[:,:,1].argmin()][0])
bottommost = tuple(cnt[cnt[:,:,1].argmax()][0])
```

For eg, if I apply it to an Indian map, I get the following result :



image

Additional Resources

Exercises

1. There are still some features left in matlab regionprops doc. Try to implement them.

Contours : More Functions

Prev Tutorial: Contour Properties
Next Tutorial: Contours Hierarchy

Goal

In this chapter, we will learn about

- Convexity defects and how to find them.
- Finding shortest distance from a point to a polygon
- Matching different shapes

Theory and Code

1. Convexity Defects

We saw what is convex hull in second chapter about contours. Any deviation of the object from this hull can be considered as convexity defect.

OpenCV comes with a ready-made function to find this, **cv.convexityDefects()**. A basic function call would look like below:

```
hull = cv.convexHull(cnt,returnPoints = False)
defects = cv.convexityDefects(cnt,hull)
```

Note

Remember we have to pass returnPoints = False while finding convex hull, in order to find convexity defects.

It returns an array where each row contains these values - **[start point, end point, farthest point, approximate distance to farthest point]**. We can visualize it using an image. We draw a line joining start point and end point, then draw a circle at the farthest point. Remember first three values returned are indices of cnt. So we have to bring those values from cnt.

```
import cv2 as cv
import numpy as np

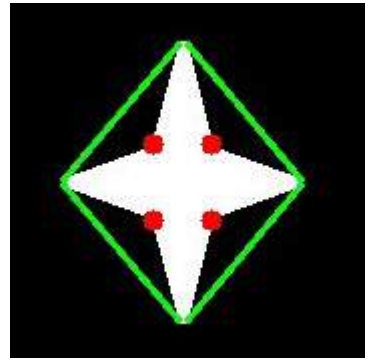
img = cv.imread('star.jpg')
assert img is not None, "file could not be read, check with os.path.exists()"
img_gray = cv.cvtColor(img,cv.COLOR_BGR2GRAY)
ret,thresh = cv.threshold(img_gray, 127, 255,0)
contours,hierarchy = cv.findContours(thresh,2,1)
cnt = contours[0]

hull = cv.convexHull(cnt,returnPoints = False)
defects = cv.convexityDefects(cnt,hull)

for i in range(defects.shape[0]):
    s,e,f,d = defects[i,0]
    start = tuple(cnt[s][0])
    end = tuple(cnt[e][0])
    far = tuple(cnt[f][0])
    cv.line(img,start,end,[0,255,0],2)
    cv.circle(img,far,5,[0,0,255],-1)

cv.imshow('img',img)
cv.waitKey(0)
cv.destroyAllWindows()
```

And see the result:



image

2. Point Polygon Test

This function finds the shortest distance between a point in the image and a contour. It returns the distance which is negative when point is outside the contour, positive when point is inside and zero if point is on the contour.

For example, we can check the point (50,50) as follows:

```
dist = cv.pointPolygonTest(cnt,(50,50),True)
```

In the function, third argument is measureDist. If it is True, it finds the signed distance. If False, it finds whether the point is inside or outside or on the contour (it returns +1, -1, 0 respectively).

Note

If you don't want to find the distance, make sure third argument is False, because, it is a time consuming process. So, making it False gives about 2-3X speedup.

3. Match Shapes

OpenCV comes with a function **cv.matchShapes()** which enables us to compare two shapes, or two contours and returns a metric showing the similarity. The lower the result, the better match it is. It is calculated based on the hu-moment values. Different measurement methods are explained in the docs.

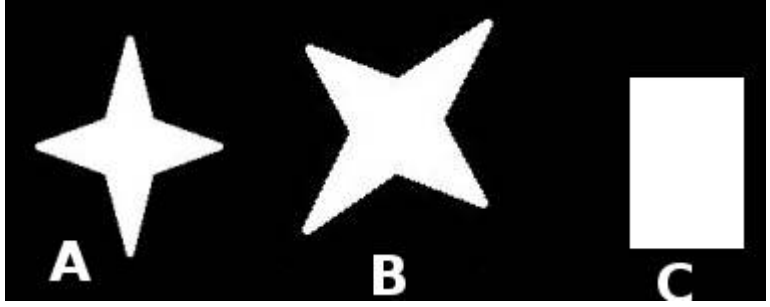
```
import cv2 as cv
import numpy as np

img1 = cv.imread('star.jpg', cv.IMREAD_GRAYSCALE)
img2 = cv.imread('star2.jpg', cv.IMREAD_GRAYSCALE)
assert img1 is not None, "file could not be read, check with os.path.exists()"
assert img2 is not None, "file could not be read, check with os.path.exists()"

ret, thresh = cv.threshold(img1, 127, 255,0)
ret, thresh2 = cv.threshold(img2, 127, 255,0)
contours,hierarchy = cv.findContours(thresh,2,1)
cnt1 = contours[0]
contours,hierarchy = cv.findContours(thresh2,2,1)
cnt2 = contours[0]

ret = cv.matchShapes(cnt1,cnt2,1,0.0)
print( ret )
```

I tried matching shapes with different shapes given below:



image

I got following results:

- Matching Image A with itself = 0.0
- Matching Image A with Image B = 0.001946
- Matching Image A with Image C = 0.326911

See, even image rotation doesn't affect much on this comparison.

Note

Hu-Moments are seven moments invariant to translation, rotation and scale. Seventh one is skew-invariant. Those values can be found using **cv.HuMoments()** function.

Additional Resources

Exercises

1. Check the documentation for **cv.pointPolygonTest()**, you can find a nice image in Red and Blue color. It represents the distance from all pixels to the white curve on it. All pixels inside curve is blue depending on the distance. Similarly outside points are red. Contour edges are marked with White. So problem is simple. Write a code to create such a representation of distance.
2. Compare images of digits or letters using **cv.matchShapes()**. (That would be a simple step towards OCR)

ENHANCED BY Google

Contours Hierarchy

Prev Tutorial: Contours : More Functions

Goal

This time, we learn about the hierarchy of contours, i.e. the parent-child relationship in Contours.

Theory

In the last few articles on contours, we have worked with several functions related to contours provided by OpenCV. But when we found the contours in image using cv.findContours() function, we have passed an argument, **Contour Retrieval Mode**. We usually passed cv.RETR_LIST or cv.RETR_TREE and it worked nice. But what does it actually mean ?

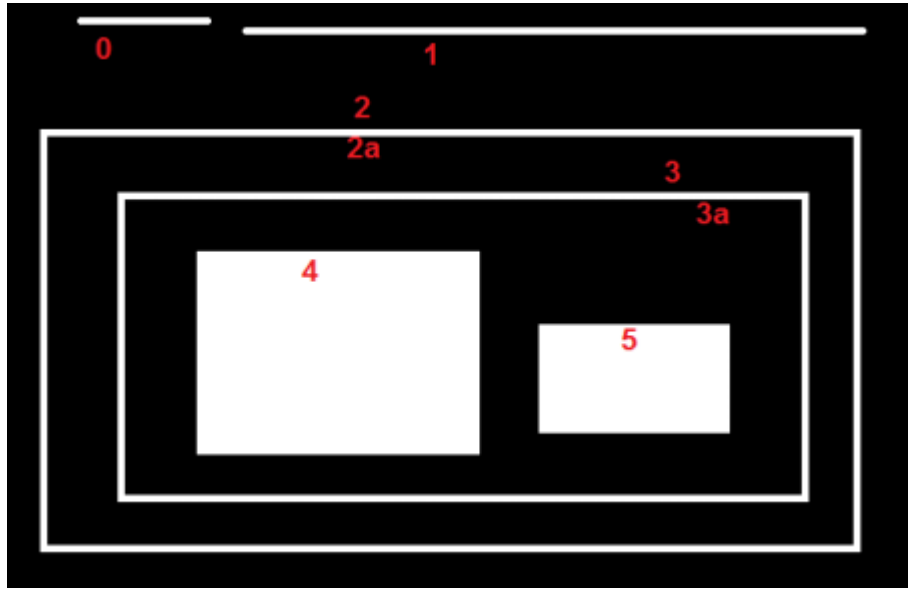
Also, in the output, we got three arrays, first is the image, second is our contours, and one more output which we named as **hierarchy** (Please checkout the codes in previous articles). But we never used this hierarchy anywhere. Then what is this hierarchy and what is it for ? What is its relationship with the previous mentioned function argument ?

That is what we are going to deal in this article.

What is Hierarchy?

Normally we use the cv.findContours() function to detect objects in an image, right ? Sometimes objects are in different locations. But in some cases, some shapes are inside other shapes. Just like nested figures. In this case, we call outer one as **parent** and inner one as **child**. This way, contours in an image has some relationship to each other. And we can specify how one contour is connected to each other, like, is it child of some other contour, or is it a parent etc. Representation of this relationship is called the **Hierarchy**.

Consider an example image below :



image

In this image, there are a few shapes which I have numbered from **0-5**. *2 and 2a* denotes the external and internal contours of the outermost box.

Here, contours 0,1,2 are **external or outermost**. We can say, they are in **hierarchy-0** or simply they are in **same hierarchy level**.

Next comes **contour-2a**. It can be considered as a **child of contour-2** (or in opposite way, contour-2 is parent of contour-2a). So let it be in **hierarchy-1**. Similarly contour-3 is child of contour-2 and it comes in next hierarchy. Finally contours 4,5 are the children of contour-3a, and they come in the last hierarchy level. From the way I numbered the boxes, I would say contour-4 is the first child of contour-3a (It can be contour-5 also).

I mentioned these things to understand terms like **same hierarchy level**, **external contour**, **child contour**, **parent contour**, **first child** etc. Now let's get into OpenCV.

Hierarchy Representation in OpenCV

So each contour has its own information regarding what hierarchy it is, who is its child, who is its parent etc. OpenCV represents it as an array of four values : **[Next, Previous, First_Child, Parent]**

"Next denotes next contour at the same hierarchical level."

For eg, take contour-0 in our picture. Who is next contour in its same level ? It is contour-1. So simply put Next = 1. Similarly for Contour-1, next is contour-2. So Next = 2.

What about contour-2? There is no next contour in the same level. So simply, put Next = -1. What about contour-4? It is in same level with contour-5. So its next contour is contour-5, so Next = 5.

"Previous denotes previous contour at the same hierarchical level."

It is same as above. Previous contour of contour-1 is contour-0 in the same level. Similarly for contour-2, it is contour-1. And for contour-0, there is no previous, so put it as -1.

"First_Child denotes its first child contour."

There is no need of any explanation. For contour-2, child is contour-2a. So it gets the corresponding index value of contour-2a. What about contour-3a? It has two children. But we take only first child. And it is contour-4. So First_Child = 4 for contour-3a.

"Parent denotes index of its parent contour."

It is just opposite of **First_Child**. Both for contour-4 and contour-5, parent contour is contour-3a. For contour-3a, it is contour-3 and so on.

Note

If there is no child or parent, that field is taken as -1

So now we know about the hierarchy style used in OpenCV, we can check into Contour Retrieval Modes in OpenCV with the help of same image given above. ie what do flags like **cv.RETR_LIST**, **cv.RETR_TREE**, **cv.RETR_CCOMP**, **cv.RETR_EXTERNAL** etc mean?

Contour Retrieval Mode

1. RETR_LIST

This is the simplest of the four flags (from explanation point of view). It simply retrieves all the contours, but doesn't create any parent-child relationship. **Parents and kids are equal under this rule, and they are just contours**. ie they all belongs to same hierarchy level.

So here, 3rd and 4th term in hierarchy array is always -1. But obviously, Next and Previous terms will have their corresponding values. Just check it yourself and verify it.

Below is the result I got, and each row is hierarchy details of corresponding contour. For eg, first row corresponds to contour 0. Next contour is contour 1. So Next = 1. There is no previous contour, so Previous = -1. And the remaining two, as told before, it is -1.

```
>>> hierarchy
array([[[ 1, -1, -1, -1],
        [ 2,  0, -1, -1],
        [ 3,  1, -1, -1],
```

```
       [ 4,  2, -1, -1],
       [ 5,  3, -1, -1],
       [ 6,  4, -1, -1],
       [ 7,  5, -1, -1],
       [-1,  6, -1, -1]]])
```

This is the good choice to use in your code, if you are not using any hierarchy features.

2. RETR_EXTERNAL

If you use this flag, it returns only extreme outer flags. All child contours are left behind. **We can say, under this law, Only the eldest in every family is taken care of. It doesn't care about other members of the family :)**.

So, in our image, how many extreme outer contours are there? ie at hierarchy-0 level?. Only 3, ie contours 0,1,2, right? Now try to find the contours using this flag. Here also, values given to each element is same as above. Compare it with above result. Below is what I got :

```
>>> hierarchy
array([[[ 1, -1, -1, -1],
       [ 2,  0, -1, -1],
       [-1,  1, -1, -1]]])
```

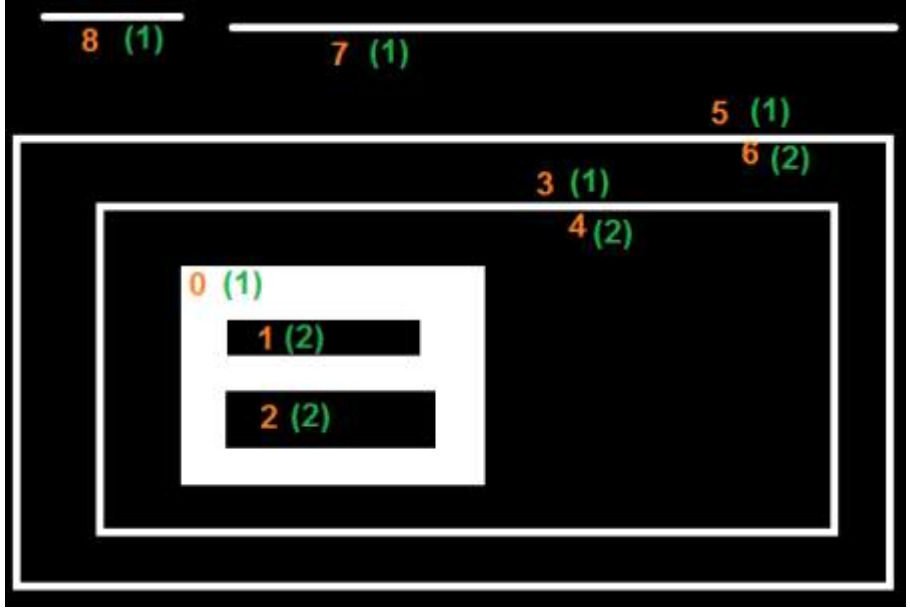
You can use this flag if you want to extract only the outer contours. It might be useful in some cases.

3. RETR_CCOMP

This flag retrieves all the contours and arranges them to a 2-level hierarchy. ie external contours of the object (ie its boundary) are placed in hierarchy-1. And the contours of holes inside object (if any) is placed in hierarchy-2. If any object inside it, its contour is placed again in hierarchy-1 only. And its hole in hierarchy-2 and so on.

Just consider the image of a "big white zero" on a black background. Outer circle of zero belongs to first hierarchy, and inner circle of zero belongs to second hierarchy.

We can explain it with a simple image. Here I have labelled the order of contours in red color and the hierarchy they belongs to, in green color (either 1 or 2). The order is same as the order OpenCV detects contours.



image

So consider first contour, ie contour-0. It is hierarchy-1. It has two holes, contours 1&2, and they belong to hierarchy-2. So for contour-0, Next contour in same hierarchy level is contour-3. And there is no previous one. And its first is child is contour-1 in hierarchy-2. It has no parent, because it is in hierarchy-1. So its hierarchy array is [3,-1,1,-1]

Now take contour-1. It is in hierarchy-2. Next one in same hierarchy (under the parenthood of contour-1) is contour-2. No previous one. No child, but parent is contour-0. So array is [2,-1,-1,0].

Similarly contour-2 : It is in hierarchy-2. There is not next contour in same hierarchy under contour-0. So no Next. Previous is contour-1. No child, parent is contour-0. So array is [-1,1,-1,0].

Contour - 3 : Next in hierarchy-1 is contour-5. Previous is contour-0. Child is contour-4 and no parent. So array is [5,0,4,-1].

Contour - 4 : It is in hierarchy 2 under contour-3 and it has no sibling. So no next, no previous, no child, parent is contour-3. So array is [-1,-1,-1,3].

Remaining you can fill up. This is the final answer I got:

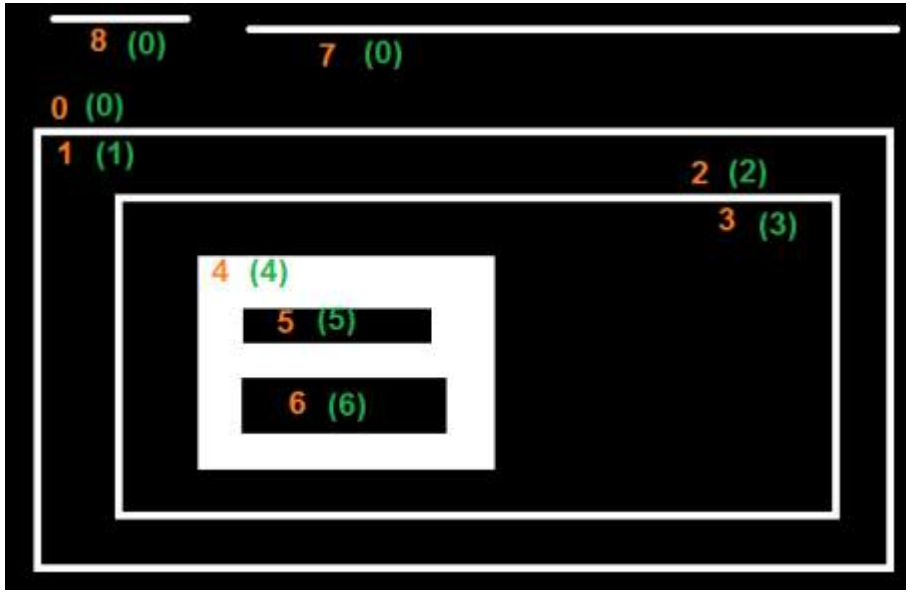
```
>>> hierarchy
array([[[ 3, -1,  1, -1],
        [ 2, -1, -1,  0],
        [-1,  1, -1,  0],
        [ 5,  0,  4, -1],
        [-1, -1, -1,  3],
```

```
       [ 7,  3,  6, -1],
       [-1, -1, -1,  5],
       [ 8,  5, -1, -1],
       [-1,  7, -1, -1]]])
```

4. RETR_TREE

And this is the final guy, Mr.Perfect. It retrieves all the contours and creates a full family hierarchy list. **It even tells, who is the grandpa, father, son, grandson and even beyond... :)**.

For example, I took above image, rewrite the code for **cv.RETR_TREE**, reorder the contours as per the result given by OpenCV and analyze it. Again, red letters give the contour number and green letters give the hierarchy order.



image

Take contour-0 : It is in hierarchy-0. Next contour in same hierarchy is contour-7. No previous contours. Child is contour-1. And no parent. So array is [7,-1,1,-1].

Take contour-2 : It is in hierarchy-1. No contour in same level. No previous one. Child is contour-3. Parent is contour-1. So array is [-1,-1,3,1].

And remaining, try yourself. Below is the full answer:

```
>>> hierarchy
array([[[ 7, -1,  1, -1],
        [-1, -1,  2,  0],
```

```
       [-1, -1,  3,  1],
       [-1, -1,  4,  2],
       [-1, -1,  5,  3],
       [ 6, -1, -1,  4],
       [-1,  5, -1,  4],
       [ 8,  0, -1, -1],
       [-1,  7, -1, -1]]])
```

Additional Resources

Exercises